Chapter 15
Using Geoprocessing Tools

ArcObjects allow developers to access the lowest level spatial and attribute data such as geometric shapes in feature classes and pixel values in raster datasets. While you can use VB programming and ArcObjects to develop custom GIS functions, tools, and applications, you can also access existing geoprocessing tools in the ArcToolbox of ArcGIS applications. Being able to accessing geoprocessing tools allows you to conveniently automate geoprocessing procedures and develop complex workflows.

This chapter will demonstrate how to access geoprocessing tools. First, it will introduce basic knowledge of geoprocessing and geoprocessing tools, particularly, how to find geoprocessing resources such as names of geoprocessing tools, tool parameters, and toolbox assemblies. Then it will show you how to instantiate tools, set up tool parameters, and execute tools. Finally, it will use examples to demonstrate the use of a number of individual geoprocessing tools, as well as how to automate spatial analysis and spatial data management using multiple geoprocessing tools.

1. Geoprocessing and Geoprocessing Tools

Geoprocessing is a process consisting of operators and tools that operate on data within ArcGIS and perform tasks that are necessary for manipulating and analyzing geographic information. Based on ArcObjects, ESRI has created many system tools and the entire geoprocessing framework. The fundamental purposes of geoprocessing are to allow GIS users to automate their GIS tasks and to perform spatial analysis and modeling.

The basic units of geoprocessing are geoprocessing tools and operators. Geoprocessing tools are programs built in ArcGIS that accomplish GIS operations such as buffering, clipping, querying, and coordinate conversion. Many tools were created based on ArcObjects. Some tools can carry out advanced spatial analysis tasks such as finding the shortest path and delineating watershed boundaries. In addition to system tools provided by the ESRI, custom tools created by the user are part of the geoprocessing tools. Typically, a geoprocessing tool processes some input data and produces a new output dataset. ESRI has created hundreds of geoprocessing tools which are organized in the ArcToolbox.

Geoprocessing tools are frequently used in desktop environment, such as ArcMap or ArcCatalog, to carry out ordinary tasks. Multiple tools can be programmed by a scripting or programming language to carry out complex tasks that involve complex workflows. Starting from ArcGIS 10.0, ESRI has adopted Python as the default geoprocessing scripting language while fully supported VB.NET for ArcObjects and geoprocessing programming.

In order to use a geoprocessing tool in a program, you must know the tool. Particularly, you must know the tool's name and parameters including input, output, as well as parameters of certain spatial operations, e.g. buffer distance. To find a tool of ArcMap, you can explore the ArcToolbox by expanding the tool trees or searching the tool name. For example, the Buffer tool exists in the tool tree: Analysis Tools >> Proximity. If you cannot find a particular tool by

navigating the tool tree in the toolbox, you can use the ArcMap search function to find it (Figure 1).

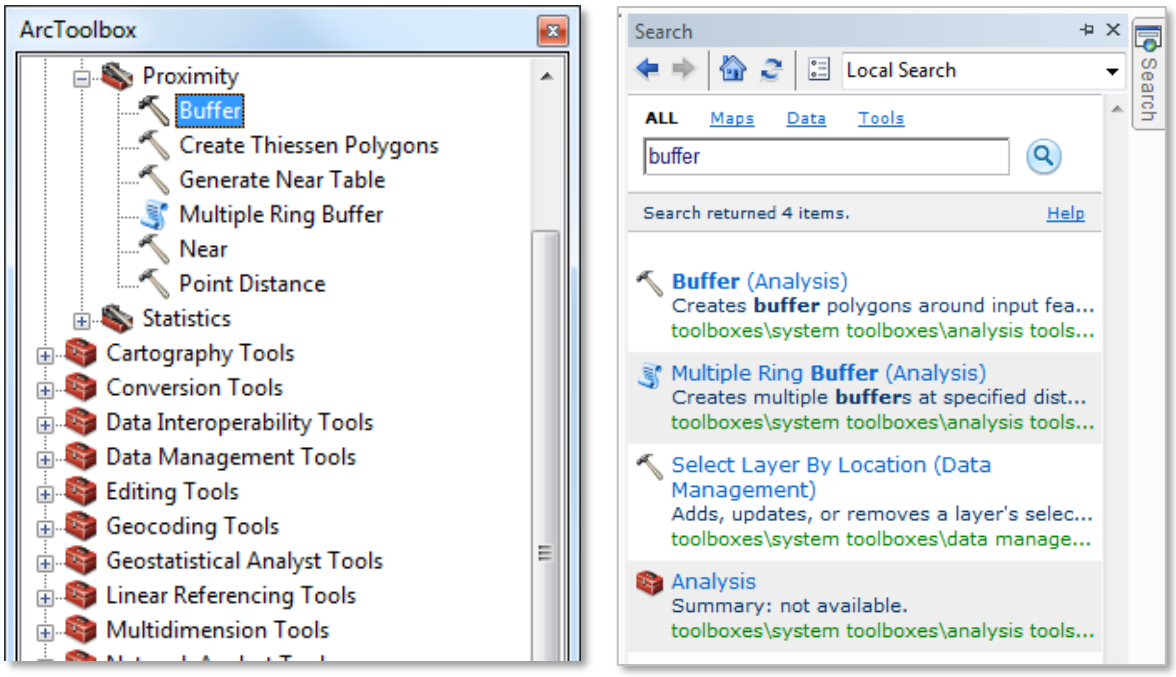


Figure 1 Finding geoprocessing tools

Once you find a tool, you can open its dialog box to find its purpose and usage, especially its parameters including requires and optional ones. Figure 2 shows the dialog box of the Buffer tool.

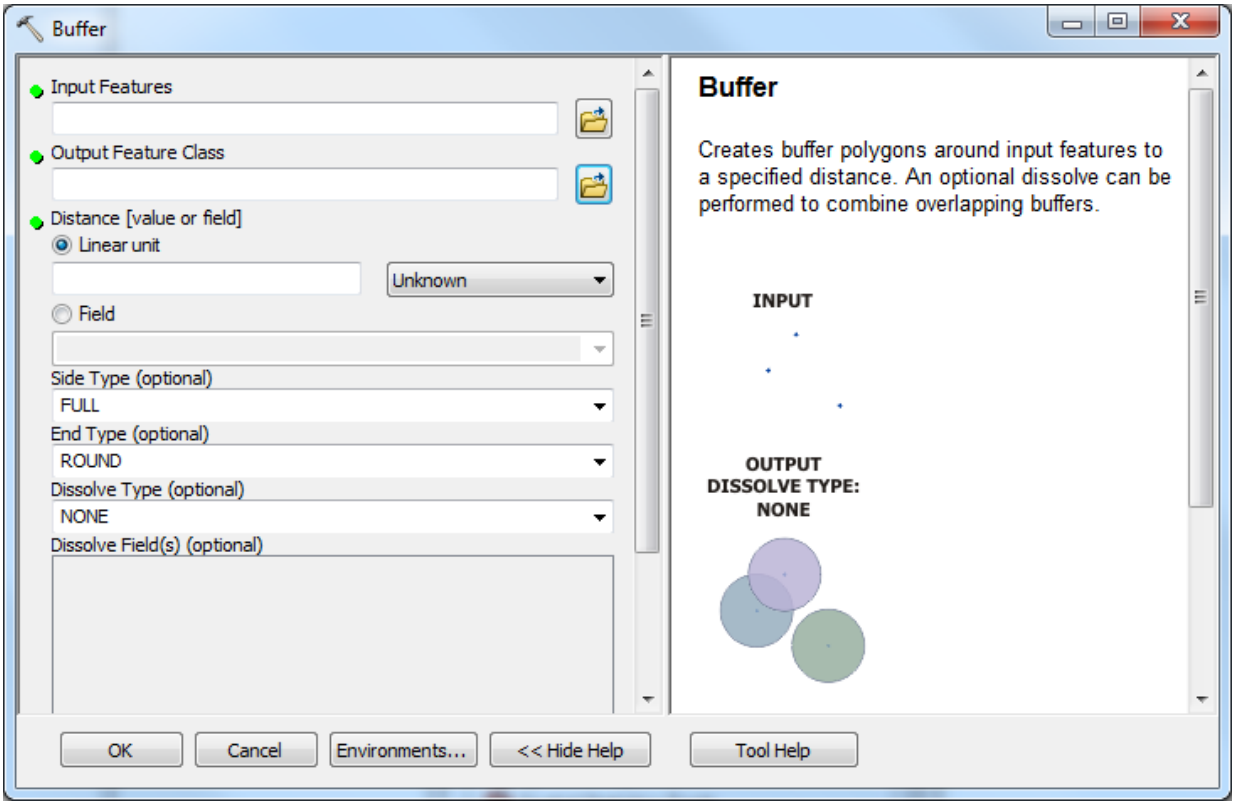


Figure 2 Finding parameters and help of the Buffer tool

The dialog box of a tool displays the tool-related information in a help panel in the right-hand side of the dialog. If the help panel does not show, you can click the "Show Help" button to open the help panel. When a tool dialog box opens, the help panel shows the purpose of the tool and briefly explains how it works. The main panel of the dialog box displays parameters of the tool. For example, Figure 2 shows that the Buffer tool has three required parameters which are marked with green dots: an input feature class, an output feature class, and a buffer distance. The buffer tool also has a few optional parameters including buffer side type, end type, dissolve type, and dissolve field(s). To get information on how to set a parameter, you can simply click on the item, then the usage of the parameter will be displayed in the help panel. When you program for geoprocessing, you must know all parameters of a geoprocessing tool to use. Whenever necessary, you can find tool information by using the tool dialog box or the online geoprocessing tool reference page at the ArcGIS Resource Center[1].

2. Toolbox assemblies

ArcGIS tools are managed by toolboxes. Each ArcGIS extension also comes with its own toolbox. In .NET, each geoprocessing system toolbox is represented by a managed assembly. You must add reference to the managed toolbox assembly in your project in order to use a tool. For example, to use the Buffer tool in the Analysis toolbox, you must add the

[1] http://resources.arcgis.com/en/help/main/10.2/index.html#/A_quick_tour_of_geoprocessing_tool_references/002t0000000z000000/

ESRI.ArcGIS.AnalysisTools assembly as a reference to your project. Table 1 shows namespaces of .NET assemblies for all system toolboxes.

Table 1 Namespaces of geoprocessing tool boxes[2]

Toolbox name	Namespace
3D Analyst tools	ESRI.ArcGIS.Analyst3DTools
Analysis tools	ESRI.ArcGIS.AnalysisTools
Conversion tools	ESRI.ArcGIS.ConversionTools
Data Management tools	ESRI.ArcGIS.DataManagementTools
Editing tools	ESRI.ArcGIS.EditingTools
Cartography tools	ESRI.ArcGIS.CartographyTools
Coverage tools	ESRI.ArcGIS.CoverageTools
Geocoding tools	ESRI.ArcGIS.GeocodingTools
Geostatistical Analyst tools	ESRI.ArcGIS.GeostatisticalAnalystTools
Linear Referencing tools	ESRI.ArcGIS.LinearReferencingTools
Multidimension tools	ESRI.ArcGIS.MultidimensionTools
Network Analyst tools	ESRI.ArcGIS.NetworkAnalystTools
Parcel Fabric tools	ESRI.ArcGIS.ParcelFabricTools
Spatial Analyst tools	ESRI.ArcGIS.SpatialAnalystTools
Spatial Statistics tools	ESRI.ArcGIS.SpatialStatisticsTools

3. Tool classes

In each toolbox assembly, there are classes representing geoprocessing tools. To run a tool, you must instantiate a tool object from a corresponding tool class as shown in the following example:

```
' to instantiate a buffer tool object, you must add the
' ESRI.ArcGIS.AnalysisTools assembly as a reference to your project
' and may need to import the namespace in the code module
Dim buf As Buffer = New Buffer(), or
Dim buf As New Buffer()
```

In VB programming, all parameters of a geoprocessing tool are treated as properties of the tool object. To pass arguments to a tool is performed by setting values to tool properties. This treatment of tool parameters takes advantage of the IntelliSense and Visual Studio IDE so that you do not have to remember all parameters of geoprocessing tools. When you type a dot (.) symbol after a tool object, VS IntelliSense immediately displays all available properties (tool parameters) and related information (Figure 3). When a property gets focused, IntelliSense also displays a brief explanation of the property. You may sometimes need to visit the online ArcGIS Resource Center or the ArcGIS Desktop help to find proper formats of values to assign to required properties.

[2] Source: http://resources.arcgis.com/en/help/arcobjects-net/conceptualhelp/index.html#/Geoprocessor_managed_assembly/0001000004r4000000/

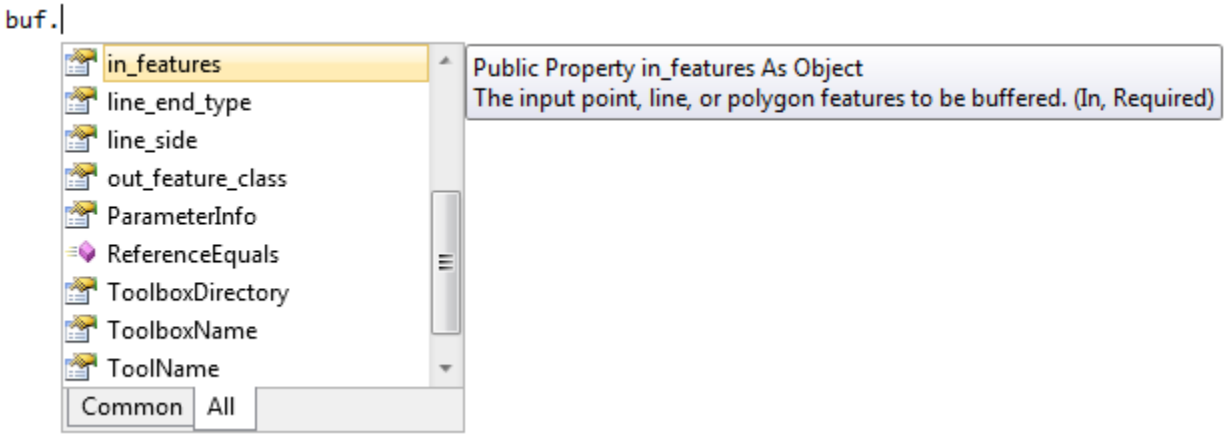


Figure 3 Properties (parameters) of the Buffer tool

A geoprocessing tool often requires multiple parameters. With Python scripting, arguments are usually passed to a tool command as a list in which the order of parameters is crucial. With VB programming, since each parameter is set as a property of the tool object, the order to set properties does not matter. You just need to make sure all required parameters are properly set. With the example of the buffer tool, you are required to set the *in_features* property (input feature class), *out_feature_class* property (output feature class), and *buffer_distance_or_field* property (buffer distance or name of a field containing buffer distances). You may optionally set the *dissolve_option* property which defines how buffers will be dissolved. The following example shows how arguments are passed to the buffer tool by setting property values:

```
buf.in_feautures = "D:\gpwks\roads.shp"
buf.out_feature_class = "D:\gpwks\roadbuf.shp"
buf.buffer_distance_or_field = 30
buf.dissolve_option = "ALL"
```

A value set to the input features property can be the full path of a shapefile (string), a feature class in a geodatabase (string), or an ArcObjects Layer object (pointing to any interface such as IFeatureLayer or ILayer) derived from an ArcMap document (hop…hop…hop…). To buffer selected features in a feature class, you can use some code to select features in ArcMap first, and then pass a layer object to the *in_features* property. To find information about parameters of a tool, please refer to the online geoprocessing tool reference at the ArcGIS Resource Center[3].

4. The Geoprocessor

To run a geoprocessing tool in a VB program, you need to use the Geoprocessor object. A Geoprocessor object can set up a geoprocessing environment and execute geoprocessing tools. A geoprocessor object can be created in two different ways. First, it can be instantiated from the traditional GeoProcessor coclass in the COM-based ESRI.ArcGIS.Geoprocessing library. The IGeoProcessor interface of the GeoProcessor class contains many methods for

[3] http://resources.arcgis.com/en/help/main/10.2/index.html#/A_quick_tour_of_geoprocessing_tool_references/002t0000000z000000/

carrying out a variety of tasks. If you have added the ESRI.ArcGIS.Geoprocessing library as a reference to your project and imported the namespace, you can use the following code to create a GeoProcessor object with IGeoProcessor interface.

```
Dim gp As IGeoProcessor = New GeoProcessor()
```

Second, the COM-based GeoProcessor co-class has been converted to a .NET class called Geoprocessor (note that the "p" is in lowercase) in newer versions of ArcGIS. The namespace of the .NET Geoprocessor assembly is ESRI.ArcGIS.Geoprocessor. When you add this assembly into your add-in project as a reference, be aware that the assembly is under the "Engine (Core)" category in the "Add ArcGIS Reference" dialog. After adding the assembly into your project and importing the namespace, the following code creates a new Geoprocessor object.

```
Dim gp As Geoprocessor = New Geoprocessor()  or
Dim gp As New Geoprocessor()
```

The new .NET-based Geoprocessor class does not use multiple interfaces so that you may use the second line to instantiate new objects. A geoprocessor objects instantiated from the .NET-based Geoprocessor class are different from an object instantiated from the COM-based GeoProcessor class. This chapter will use the .NET-based Geoprocessor class and you are encouraged to use it. For more information about the Geoprocessor class, please refer to the online Geoprocessor member reference[4].

4.1 Setting Geoprocessing Environment

Once you have created a Geoprocessor object, you can use it to set a geoprocessing environment. For example, you can specify a workspace in which input datasets are located and output datasets will be saved. You can also set a geoprocessing extent and specify a coordinate system for output datasets, etc.

Setting geoprocessing environment is accomplished by calling the SetEnvironmentValue(*name*, *value*) method of a geoprocessor object. This method requires two arguments: an environment name and an environment value, both are string values. For example,

```
gp.SetEnvironmentValue("workspace", "D:\Data\gpwks")
gp.SetEnvironmentValue("extent", "MBR.shp")
gp.SetEnvironmentValue("outputCoordinateSystem",
                       "D:\gpwks\NAD_1927_UTM_Zone_12N.prj")
```

Environment names are not arbitrary strings. You must enter a valid name in order to set an environment successfully. Table 2 shows all geoprocessing environment names in alphabetical order. The first column in the table lists the name of the environment. When you set an environment, you must pass a name as a string (by putting it inside a pair of double quotation marks) to the geoprocessor's SetEnvironmentValue() method. The second column contains display names shown in the Environment Settings page of the tool dialog box. On the ArcGIS Resources web page (URL in footnote), each environment display name in the table is linked to

---

4 http://help.arcgis.com/en/sdk/10.0/arcobjects_net/componenthelp/index.html#/Members/004700002t2p000000/

the reference page of that environment, which explains the purpose of the environment and permissible values.

Table 2 Geoprocessing environment names[5]

Environment name	**Display name**
autoCommit	Auto Commit
cartographicCoordinateSystem	Cartographic Coordinate System
cellSize	Cell size
coincidentPoints	Coincident points
compression	Compression
configKeyword	Output CONFIG Keyword
derivedPrecision	Precision For Derived Coverages
extent	Extent
geographicTransformations	Geographic Transformations
maintainSpatialIndex	Maintain Spatial Index
mask	Mask
MDomain	Output M Domain
MResolution	M Resolution
MTolerance	M Tolerance
newPrecision	Precision For New Coverages
outputCoordinateSystem	Output Coordinate System
outputMFlag	Output has M Values
outputZFlag	Output has Z values
outputZValue	Default output Z value
projectCompare	Level Of Comparison Between Projection Files
pyramid	Pyramid
qualifiedFieldNames	Maintain fully qualified field names
randomGenerator	Random number generator
rasterStatistics	Raster statistics
referenceScale	Reference Scale
scratchWorkspace	Scratch Workspace
snapRaster	Snap Raster
spatialGrid1, 2, 3	Output Spatial Grid 1, 2, 3
terrainMemoryUsage	Terrain Memory Usage
tileSize	Tile size
tinSaveVersion	TIN storage version
workspace	Current Workspace
XYDomain	Output XY Domain
XYResolution	XY Resolution
XYTolerance	XY Tolerance
ZDomain	Output Z Domain

[5] Source: http://resources.arcgis.com/en/help/arcobjects-net/conceptualhelp/index.html#/Using_environment_settings/0001000001n5000000/

ZResolution	Z Resolution
ZTolerance	Z Tolerance

Values set to geoprocessing environments by the SetEnvironmentValue() method are also strings. Depending on a specific environment, values may be the full path of a file such as shapefile or projection file, the name of a feature class in a geodatabase, a block of valid text, or some quoted numbers. For example, to set a workspace, you may specify the full path of a folder, the path of a personal or a file geodatabase, or constant "in_memory". To set a geoprocessing extent, you may specify a shapefile name, a feature class name, or coordinates of the four limits of a rectangle (minx, miny, maxx, maxy). To set an output coordinate system, you may provide the full path of a projection file (with extension .prj), a block of text representing the content of a .prj file, or a shapefile or feature class projected to a desired coordinate system. See examples below.

```
    gp.SetEnvironmentValue("workspace", "D:\gpwks\meniscus.mdb")
    gp.SetEnvironmentValue("extent", "443912.89, 3881725.38, 447295.47, 3885107.95")
    gp.SetEnvironmentValue("outputCoordinateSystem",
    "PROJCS['NAD_1927_UTM_Zone_12N',GEOGCS['GCS_North_American_1927',DATUM['D_North_Ameri
can_1927',SPHEROID['Clarke_1866',6378206.4,294.9786982]],PRIMEM['Greenwich',0.0],UNIT['De
gree',0.0174532925199433]],PROJECTION['Transverse_Mercator'],PARAMETER['False_Easting',50
0000.0],PARAMETER['False_Northing',0.0],PARAMETER['Central_Meridian',-
111.0],PARAMETER['Scale_Factor',0.9996],PARAMETER['Latitude_Of_Origin',0.0],UNIT['Meter',
1.0]]")
```

As you can see, a same environment can be set with different type of values. In geoprocessing, since an output dataset takes the coordinate system of input datasets by default, the "outputCoordinateSystem" parameter is optional unless you want the output to have a different coordinate system. Setting a long string to the "outputCoordinateSystem" parameter in the above example is only to demonstrate a possibility. Normally, you can set a projection file to the parameter when necessary. If a projection file is not available in situations such as when you want to use the spatial reference of a feature dataset or feature class in a geodatabase, you can create one using ArcCatalog. To do so, right click on a feature dataset or feature class, then select Properties >> XY Coordinate System >> Save As.

4.2 Executing geoprocessing tools

Once you have instantiated a geoprocessor object, properly set up a geoprocessing environment, instantiated a geoprocessing tool and properly set its properties, you can call the Execute(*GeoProcess*, *TrackCancel*) method of the geoprocessor object to execute the tool.

The Execute() method requires two arguments. The first argument asks for a geoprocessing tool object pointing to the IGeoProcess interface. Since all geoprocessing tool classes inherit the IGeoProcess interface, you can pass any well set geoprocessing tool object, such as a buffer object, to this argument to execute.

The second argument needs a CancelTracker object with ITrackCancel interface (included in namespace ESRI.ArcGIS.System). A CancelTracker object is used by ArcObjects to monitor any operation, such as pressing the Esc key, the space bar, and mouse clicking, performed by the user during a process, and to determine whether to terminate processes. A CancelTracker object is typically needed by functions that execute a lengthy operation such as, printing, exporting and drawing.

The ITrackCancel interface provides access to properties and methods that determine if a cancellation has been requested by the user, and it also allows developers to specify what actions constitute a cancellation (Figure 3).

Table 3 Members of the ITrackCancel interface[6]

	Cancel	Cancels the associated operation.
	CancelOnClick	Indicates whether mouse clicks should cancel the operation.
	CancelOnKeyPress	Indicates whether the escape key and spacebar should cancel the operation.
	CheckTime	The interval at which the operation will be interrupted to advance progressors and process messages.
	Continue	Called frequently while associated operation is progressing. A return value of false indicates that the operation should stop.
	ProcessMessages	An obsolete method.
	Progressor	The progressor used to show progress during lengthy operations.
	Reset	Resets the manager after the associated operation is finished.
	StartTimer	An obsolete method.
	StopTimer	An obsolete method.
	TimerFired	An obsolete method.

To enable the user to interrupt the execution of a geoprocessing tool that takes a long time, you can instantiate a CancelTracker object from the CancelTrack coclass (included in assembly ESRI.ArcGIS.Display), properly set up its properties and pass it to the second parameter of the Execute() method. If you do not want to interrupt the execution process (true in most cases) or you are not sure how to set up a CancelTracker object, you can always pass the Nothing object to the second parameter of the Execute() method. For example, the following sample code executes a buffer tool.

```
gp.Execute(buf, Nothing)        ' execute the buffer tool
```

To enable cancelling the execution of the buffer tool by using the Esc key, you can use the following code.

```
Dim tc As ESRI.ArcGIS.esriSystem.ITrackCancel = New ESRI.ArcGIS.Display.CancelTracker
gp.Execute(buf, tc)
```

4.3 Monitoring geoprocessing execution status

The Execute() method of the Geoprocessor class returns a GeoProcessorResult object with IGeoProcessorResult interface. This object contains messages of execution process such as number of inputs, outputs, cancellation, and success status etc. The IGeoProcessorResult interface exits in the COM-based ESRI.ArcGIS.Geoprocessing library, so you need to add this library into your project as a reference as well in order to use the GeoProcessorResult object. For example,

[6] Source: http://help.arcgis.com/en/sdk/10.0/arcobjects_net/componenthelp/index.html#//00420000022q000000

```
Dim result As ESRI.ArcGIS.Geoprocessing.IGeoProcessorResult2
result = gp.Execute(buf, tc)

If result.Status = ESRI.ArcGIS.esriSystem.esriJobStatus.esriJobSucceeded Then
    MsgBox("Buffering completed successfully.")
Else
    MsgBox("Buffering was not completed.")
End If
```

5. Application

This section uses examples to demonstrate VB programming for several geoprocessing tools in three toolsets: analysis tools, spatial analysis tools, and data management tools. To get started, please copy folder "Data\gpwks" from the CD-ROM to your hard drive (e.g. D:\Data\gpwks), you will use this folder as your geoprocessing workspace. Then create a new ArcMap add-in project named "GPTools" in VB (use icon  for the project), and create the following Add-in buttons and a toolbar (use caption "Geoprocessor Toolbar") to hold these buttons. You can find all icons in the Images folder on the CD-ROM.

Button Name	Icon	Caption/Tooltip Text
DataButton		Data Management
SelectButton		Select Features
ClipButton		Clip Analysis
IdentityButton		Overlay Analysis
BufferButton		Buffer Analysis
SlopeButton		Slope Creation
ContourButton		Contour Creation

After creating the new Add-in project with the tools and toolbar, please add the following assemblies into the project as references:

ESRI.ArcGIS.Carto
ESRI.ArcGIS.Geoprocessor

Depending on the geoprocessing tools and toolboxes to use, you will add more assemblies into the add-in project as references. Refer to Table 1 for geoprocessing toolboxes and assembly namespaces.

5.1 Analysis Tools

The ESRI.ArcGIS.AnalysisTools assembly provides ordinary vector-based spatial analysis tools and basic statistical tools. This section will allow you to create and use of the Select tool, Clip tool, Identity tool, and Buffer tool. In order to use the analysis toolbox, please add the ESRI.ArcGIS.AnalysisTools .NET assembly into your project as a reference.

5.1.1 Select tool

To demonstrate the Select tool, you will implement the SelectButton add-in button to make it select limestone terrain (including limestone rocks and limestone residual soil) from the geology shapefile and save result as a shapefile named “Limestone.shp” in the workspace. After adding necessary references to the project, please open the SelectButton class module and import the following namespaces for coding convenience:

```
Imports ESRI.ArcGIS.Geoprocessor
Imports ESRI.ArcGIS.AnalysisTools
```

Then please enter the following code in the OnClick() event procedure:

```
    ' instantiate a Geoprocessor object
    Dim gp As Geoprocessor = New Geoprocessor()

    ' set workspace environment, make sure the workspace is correct
    gp.SetEnvironmentValue("workspace", "D:\Data\gpwks")

    ' to allow overwriting output
    gp.OverwriteOutput = True

    ' instantiate a Select tool (object)
    Dim sel As [Select] = New [Select]()

    sel.in_features = "Geology.shp"              ' specifies input dataset
    sel.out_feature_class = "Limestone.shp"      ' specifies output dataset
    sel.where_clause = "Code = 8 OR Code = 12 OR Code = 14"    ' selection criteria

    gp.Execute(sel, Nothing)        ' execute the tool
```

This is really simple! After executing this tool, a resulting limestone layer will be automatically added into the map. I would like to draw your attention to a couple of lines in the code. First, code `gp.OverwriteOutput = True` is to allow the program to overwrite output dataset (Limestone shapefile in this case) without warning. This setting is especially convenient when you repeatedly test the program since you do not have to manually delete the output shapefile before executing the tool again. Second, the line that instantiates the select tool, the name of the Select class is placed inside a pair of square brackets. This is because the class name “Select” conflicts a built-in keyword in the VB programming language. The brackets are automatically added by IntelliSense. If you refer the Select class with its full namespace, the brackets are not necessary, e.g.

```
    Dim sel As New ESRI.ArcGIS.AnalysisTools.Select()
```

In this example, the input and output datasets are specified as shapefiles in the same workspace set in the geoprocessing environment. If you want to use a shapefile that does not exist in the specified workspace as input, you can set the full path of the file to the in_features property.

This tool should work if the workspace and the input shapefile exist and the input shapefile has a field named Code. To make the program strong, you can use additional code to validate all user inputs.

5.1.2 Clip tool

You will implement the ClipButton add-in button to make it clip shapefile Roads.shp using shapefile MBR.shp in the workspace. The process will be very similar to the implementation of the select button. You will instantiate a geoprocessing tool object from the Clip class and properly set its properties. Please open the ClipButton class module and add the same import statements as the Select tool. Then enter the following code in the OnClick event procedure of the clip add-in button:

```
Dim gp As Geoprocessor = New Geoprocessor()
gp.SetEnvironmentValue("workspace", "D:\Data\gpwks")    ' make sure folder exists
gp.OverwriteOutput = True

Dim clip As New Clip()
clip.in_features = "Roads.shp"
clip.out_feature_class = "Road_clip.shp"
clip.clip_features = "MBR.shp"

gp.Execute(clip, Nothing)
MsgBox("Clip done!")
```

5.1.3 Identity tool

Identity is a type of overlay analysis in which features of an input layers are intersected by an identity layer. The output layer takes the extent of the first input layer and features in the output layer carry attributes of both layers. Since the Identity tool also exists in the AnalysisTools toolbox, to implement this tool is very similar to the above two examples. The main differences lie in setting tool parameters.

This example will let you to implement the IdentityButton add-in button by instantiating an identity geoprocessing tool object from the Identity class and using it to overlay the canopy feature class and soil feature class in the Meniscus file geodatabase in the workspace. Please import the Geoprocessor and AnalysisTools namespaces into the IdentityButton class module and enter the following code in OnClick event procedure.

```
Dim gp As New Geoprocessor()
gp.SetEnvironmentValue("workspace", "D:\Data\gpwks\Meniscus.gdb")
gp.OverwriteOutput = True

Dim identity As New Identity()
identity.in_features = "Canopy"
identity.identity_features = "Soil"
identity.out_feature_class = "Output/Cansoil"

gp.Execute(identity, Nothing)
MsgBox("Identity done!")
```

One thing to note in the code above is that the workspace is a file geodatabase. Although the input and identity feature classes are in a feature dataset (UTM27), the feature dataset name are not required because each feature class is uniquely identified in a geodatabase. However, since the resulting feature class “Cansoil” needs to be stored in feature dataset “Output”, the output dataset does need to be specified in the out feature class string.

5.1.4 Buffer tool

Suppose a water source protection project needs to create a series of buffer zones around a water body, you are going to create a buffer tool to carry out the task. The tool should provide a friendly GUI to allow the user to specify analysis parameters and set up an analysis environment, and it should output one shapefile consisting of a series of buffer polygons clipped to the project area.

To implement this buffer tool, let's start to create a Windows form in your project as a dialog box for user interaction (Figure 4). The form should allow the user to select a layer from a list of loaded layers as input, to select a layer as clip layer, to specify a workspace and output shapefile name, as well as to select an attribute of the selected layer and specify an attribute value so as to define an expression for selecting features (the water body) to buffer. The form should also allow the user to specify the number of buffer zones and buffer intervals along with other options. Please design the GUI of the form by referring to Figure 4 and the table below.

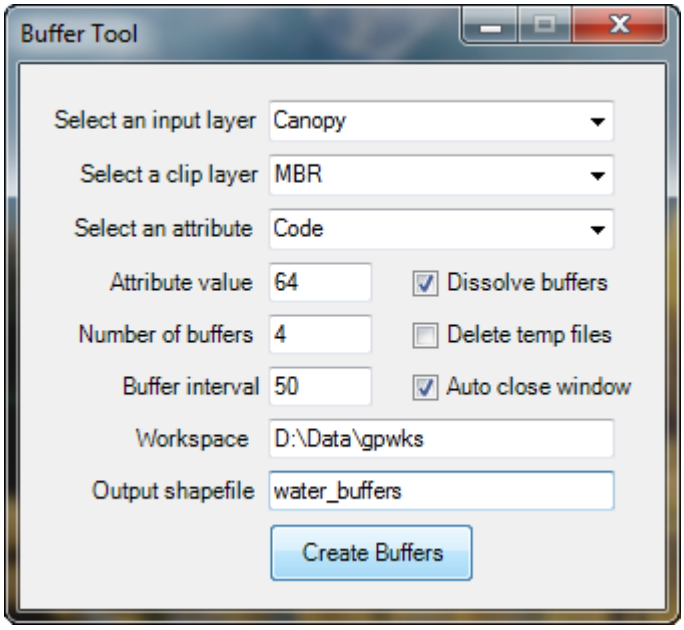


Figure 4 The buffer tool user interface

Control	Name	Property Setting
Form	frmBuffer	Text: Buffer Tool; MaximizeBox: False; FormBorderStyle: FixedDialog
ComboBox	cboInLayer	TabIndex: 0
ComboBox	cboClipLayer	TabIndex: 1
ComboBox	cboFields	TabIndex: 2
TextBox	txtAttVal	TabIndex: 3
TextBox	txtBufNum	TabIndex: 4; Text: 4
TextBox	txtInterval	TabIndex: 5; Text: 50
CheckBox	chkDissolveBuffers	TabIndex: 6; Text: Dissolve buffers; Checked: True
CheckBox	chkDelTempFiles	TabIndex: 7; Text: Delete temp files; Checked: False
CheckBox	chkAutoClose	TabIndex: 8; Text: Auto close window; Checked: True
TextBox	txtWorkspace	TabIndex: 9; Text: D:\Data\gpwks

TextBox	txtOutShapefile	TabIndex: 10; Text: water_buffers
Button	btnCreateBuffers	TabIndex: 11; Text: Create Buffers

After creating the form, use the following code in the OnClick event procedure of the add-in button (BufferButton) to open the form.

```
Dim frmBuffer As frmBuffer = New frmBuffer()
frmBuffer.ShowDialog()
```

Run the program to test the buffer button to see if you can open the dialog box. If it works, then close the window and stop the program to implement the main functionality in the form class. To do so, add the Geodatabase assembly into the project as a reference, and import the following namespaces in the frmBuffer class module.

```
Imports ESRI.ArcGIS.ArcMapUI      ' needed for IMxDocument
Imports ESRI.ArcGIS.Carto         ' needed for IMap and ILayer etc.
Imports ESRI.ArcGIS.Geodatabase   ' needed for IFields, IField, and IQueryFilter etc.
Imports ESRI.ArcGIS.Geoprocessor  ' needed for Geoprocessor
Imports ESRI.ArcGIS.AnalysisTools ' needed for analysis tools (select, clip, buffer)
```

Populate the two combo boxes with layers already loaded into ArcMap in the Load event of the Windows form, so that the user can select an input layer and a clip layer when the dialog shows.

```
Private Sub Buffer_Form_Load(...) Handles MyBase.Load
    Dim mxDoc As IMxDocument = My.ArcMap.Document     ' gets the map document
    Dim map As IMap = mxDoc.FocusMap                  ' gets the focus map
    For i As Integer = 0 To map.LayerCount - 1        ' iterates through layers
        If TypeOf map.Layer(i) Is IFeatureLayer Then  ' if it is a feature layer
            cboInLayer.Items.Add(map.Layer(i).Name)   ' adds it into cboInLayer
            cboClipLayer.Items.Add(map.Layer(i).Name) ' adds it into cboClipLayer
        End If
    Next
End Sub
```

Inside the repetition block, a VB operator “TypeOf … Is” is used to check if a layer is a feature layer because we want to exclude raster layers from combo boxes. The TypeOf … Is operator tests whether an object is compatible with a data type (or interface) by putting the object in between the two key words and a data type (interface) after the operator. The result of this operation is a Boolean value True or False. In the code above, if a layer object (map.Layer(i)) is compatible with interface IFeatureLayer, the result of the operation is True and the layer name is added into the combo boxes.

Now, you can run the program to test the dialog. After ArcMap opens, please add some layers into the map. You can add some feature layers and some raster layers as well. Then click the buffer button to open the dialog and check if the combo boxes (cboInLayer and cboClipLayer) are populated. If successful, stop the program and continue to work on the implementation.

Next, we hope that when the user selects an input layer in the cboInLayer combo box, the cboFields combo box is populated with field names of the selected layer. This task can be carried out in the SelectedIndexChanged event procedure of the cboInLayer combo box. Please implement the SelectedIndexChanged event using the following is the code.

```
Private Sub cboInLayer_SelectedIndexChanged(...) _
            Handles cboInLayer.SelectedIndexChanged
    If cboInLayer.SelectedIndex < 0 Then Return ' no item is selected
    ' get the layer object with name specified by the user
    Dim flyr As IFeatureLayer = GetLayerByName(cboInLayer.Text)
    ' get the fields object from the selected feature layer
    Dim fields As IFields = flyr.FeatureClass.Fields

    cboFields.Items.Clear()                          ' clears the combo box
    For i As Integer = 0 To fields.FieldCount - 1    ' iterates through fields
        If fields.Field(i).Name <> "Shape" Then      ' skips the Shape field
            cboFields.Items.Add(fields.Field(i).Name)  ' adds a field name into box
        End If
    Next
End Sub
```

The procedure first checks if there is a user selected item in the cboInLayer combo box. If not, the SelectedIndex property of the cboInLayer control is -1. Otherwise, it gets a layer object with IFeatureLayer interface by calling a custom function named GetLayerByName(). The GetLayerByName() function takes a layer name as input and returns a Layer object pointing to IFeatureLayer interface. The following is the code for function GetLayerByName():

```
Private Function GetLayerByName(LayerName As String) As IFeatureLayer
    Dim mxDoc As IMxDocument = My.ArcMap.Document
    Dim map As IMap = mxDoc.FocusMap
    Dim lyr As ILayer = Nothing
    For i As Integer = 0 To map.LayerCount - 1
        If TypeOf map.Layer(i) Is IFeatureLayer And _
           map.Layer(i).Name = LayerName Then
            lyr = map.Layer(i)
            Exit For
        End If
    Next
    Return lyr
End Function
```

The TypeOf … Is operator is used again inside a repetition block to check if each layer in the active map is a feature layer. After adding the above code into the frmBuffer form class, you can start the program to test if the cboFields combo box is populated when you select a layer in the cboInLayer combo box. If successful, you can close ArcMap and move on.

After implementing the combo boxes, you are ready to implement the Click event of the “Create Buffers” button in the dialog window. You will let this event procedure to carry out the buffering task in eight steps:

1) Validate user input
2) Initialize a geoprocessing environment. This includes instantiating a geoprocessor object, setting up a workspace, and creating some file names for intermediate and output shapefiles.
3) Get a layer object from the map document based on the user selected layer name (suppose “Canopy” is selected)

4) Select features from the layer to buffer based on user specified criteria. Suppose the user selects attribute “Code” and provides value 64 (water), expression “Code = 64” can be used to select water bodies in the canopy feature class.
5) Create a series of buffer zones using various buffer distances.
6) Union multiple buffer shapefiles into one shapefile.
7) Clip the combined buffer shapefile by the MBR shapefile to produce final output.
8) Delete temporary files depending on user selection.

You will create a sub procedure or a function to accomplish each step listed above. After you create one custom procedure, you can call it in the Click event procedure of the button (btnCreateBuffers) to test it. Finally, when all necessary procedures are created and are called sequentially in the Click event procedure, your code will look beautiful.

First, let’s start with creating a function named ValidateUserInput() as the following.

```
Private Function ValidateUserInput() As Boolean
    If cboInLayer.SelectedIndex < 0 Then Return False
    If cboClipLayer.SelectedIndex < 0 Then Return False
    If cboFields.SelectedIndex < 0 Then Return False
    If txtAttVal.Text = "" Then
        MsgBox("Please enter an attribute value", MsgBoxStyle.Critical)
        Return False
    End If
    If Not IsNumeric(txtBufNum.Text) Then
        MsgBox("Please enter a valid number of buffers.", MsgBoxStyle.Critical)
        Return False
    End If
    If Not IsNumeric(txtInterval.Text) Then
        MsgBox("Please enter a valid buffer interval.", MsgBoxStyle.Critical)
        Return False
    End If
    If Not IO.Directory.Exists(txtWorkspace.Text) Then
        MsgBox("The workspace does not exist.", MsgBoxStyle.Critical)
        Return False
    End If
    If txtOutShapefile.Text = "" Then
        MsgBox("Please enter an output shapefile name.", MsgBoxStyle.Critical)
        Return False
    End If
    Return True
End Function
```

Step 2, let’s create a sub procedure InitializeGeoprocessing() that instantiates a geoprocessor object and sets up its parameters. This procedure will also create some file names for temporary shapefiles and the final output shapefile. Because the geoprocessor object and the file names will be used in other procedures in the frmBuffer class, variables for the geoprocessor and file names must be declared as members of the class (i.e. class-level variables). Please declare the following variables immediately below the class declaration line (`Public Class frmBuffer`):

```
Private gp As Geoprocessor      ' the Geoprocessor object
Private tmpBufs() As String     ' temporary shapefile names for various buffer sizes
Private bufUnion As String      ' temporary shapefile name for united buffers
```

```
    Private outShapefile As String  ' output shapefile name
```

The following is the code for the InitializeGeoprocessing() procedure:

```
    Private Sub InitializeGeoprocessing()
        gp = New Geoprocessor       ' instantiates the gp object
        gp.SetEnvironmentValue("workspace", txtWorkspace.Text)  ' sets a workspace
        gp.OverwriteOutput = True   ' allows overwrite output

        Dim num As Integer = CInt(txtBufNum.Text)       ' gets number of buffers
        Dim intvl As Integer = CInt(txtInterval.Text)   ' gets buffer interval
        Dim bufSize As Integer
        ReDim tmpBufs(num - 1)      ' specifies an upperbound for array tmpBufs

        ' create a series of file names for buffer shapefiles, one for a size
        For i As Integer = 1 To num
            bufSize = i * intvl
            tmpBufs(i - 1) = "buf" & bufSize.ToString & ".shp"
        Next

        bufUnion = "bufUnion.shp"       ' union buffer shapefile name

        ' standardize the user entered output shapefile name
        If txtOutShapefile.Text.IndexOf(".shp") > 0 Then
            ' if the user-provided shapefile name contains extension .shp
            outShapefile = txtOutShapefile.Text
        Else
            ' the user-provided shapefile name does not contain extension .shp
            outShapefile = txtOutShapefile.Text & ".shp"    ' add an extention
        End If
    End Sub
```

Step 3, to get a layer object from the map document, you can call the GetLayerByName() function that has been created in the class.

Step 4, you will create a new function named SelectFeatures() that selects features from a feature layer based on user specified attribute and value. The function has three parameters: a layer object with IFeatureLayer interface, a field name as string and a value as also string, and it returns an integer number indicating the number of features selected. Please recall that no matter what method is used to select features from a feature layer, resulting selected features are hold by a selection set object. If necessary, you can refer to object model diagrams in Chapter 12.

```
    Private Function SelectFeatures(Layer As IFeatureLayer,
                                    FieldName As String, Value As String) As Integer
        ' get the fields object from the feature class
        Dim fields As IFields = Layer.FeatureClass.Fields
        ' get the index of the user specified field
        Dim x As Integer = fields.FindField(FieldName)
        If x < 0 Then Return -1   ' if invalid field name encountered

        ' create a selection expression
        Dim expr As String
        If fields.Field(x).Type = esriFieldType.esriFieldTypeString Then
```

```
            ' if the field is a string type field, the value must be quoted
            expr = FieldName & " = '" & Value & "'"
        Else
            ' if the field is numeric, the value does not need to be quoted.
            ' for simplicity, all non-string type field are treated as numeric here
            ' you may add more code to make the program perfect
            expr = FieldName & " = " & Value
        End If

        Dim fs As IFeatureSelection = Layer          ' QI to IFeatureSelection interface
        Dim qf As IQueryFilter = New QueryFilter()  ' creates a query filter object
        qf.WhereClause = expr                       ' sets a selection criterion

        ' finally, perform select features
        Try      ' use a try block to avoid crashing caused by invalid query expression
            fs.SelectFeatures(qf, esriSelectionResultEnum.esriSelectionResultNew, False)
        Catch ex As Exception

        End Try
        My.ArcMap.Document.ActiveView.Refresh()

        Dim selset As ISelectionSet = fs.SelectionSet   ' gets the selection set
        Return selset.Count     ' returns the number of selected features
    End Function
```

Step 5, this is the core part of this tool. You will create a sub procedure CreateBuffers() that uses the Buffer tool in the AnalysisTools assembly to create a series of buffers around the selected features using a user-specified size interval. This procedure will take four arguments: an input feature layer object (containing selected features to buffer), the number of buffers to create, a buffer size interval, and an optional Boolean argument that indicates whether to dissolve overlapping buffer polygons. Before entering the following procedure into your project, please add the Display assembly into the project as a reference.

```
    Private Sub CreateBuffers(Layer As IFeatureLayer,
                              BufferNumber As Integer,
                              BufferInterval As Integer,
                              Optional DissolveBuffers As Boolean = True)

        My.ArcMap.Application.StatusBar.Message(0) = "Creating buffers..."

        gp.OverwriteOutput = True       ' allows overwrite output
        gp.AddOutputsToMap = False      ' suppresses display of temporary output

        Dim buf As New Buffer()         ' instantiates a buffer tool object
        buf.in_features = Layer         ' sets input feature layer

        If DissolveBuffers Then
            buf.dissolve_option = "ALL"     ' sets dissolve buffer option
        End If

        ' to enable cancellation of execution of the tool by the Esc key
        Dim tc As ESRI.ArcGIS.esriSystem.ITrackCancel =
            New ESRI.ArcGIS.Display.CancelTracker

        Dim BufferSize As Integer
        For num As Integer = 1 To BufferNumber          ' iterates each buffer size
```

```
            BufferSize = num * BufferInterval           ' calculates a buffer size
            buf.buffer_distance_or_field = BufferSize   ' sets buffer size
            buf.out_feature_class = tmpBufs(num - 1)    ' sets output shapefile
            gp.Execute(buf, tc)                         ' executes the buffer tool
        Next

        buf = Nothing
        My.ArcMap.Application.StatusBar.Message(0) = "Buffers created."

        My.ArcMap.Application.CurrentTool = Nothing
    End Sub
```

The DissolveBuffers parameter of the procedure is declared with keyword Optional. An optional parameter must be assigned a default value at declaration. When the CreateBuffers() procedure is called, the client may or may not provide a value to this parameter. If no value is passed to an optional parameter, the default value will be used.

While setting an input to the buffer object in the above procedure, a layer object pointing to IFeaturelayer interface is set to the in_features property. This is different from the previous two examples (select and clip) in which a shapefile name is assigned to the in_features property of the tools.

In the middle of the CreateBuffers() procedure, a CancelTracker object is created. This object is passed to the second parameter of the Execute() method of the geoprocessor object to enable cancellation of the execution by pushing the Esc key. The CancelTracker class is included in assembly ESRI.ArcGIS.Display, but interestingly, the ICancelTracker interface is provided by the ESRI.ArcGIS.System assembly.

Step 6, create a sub procedure named UnionBuffers() to merge the separate buffer shapefiles into one shapefile.

```
    Private Sub UnionBuffers()
        My.ArcMap.Application.StatusBar.Message(0) = "Union buffers..."
        gp.AddOutputsToMap = False      ' suppresses display of temporary results

        Dim union As New Union()        ' instantiates a Union object

        ' create a semi-colon delimited input shapefile name list
        Dim shapeList As String = ""
        For i As Integer = 0 To tmpBufs.Length - 1
            shapeList &= tmpBufs(i) & "; "
        Next
        ' trim off the last ";" and blank space
        shapeList = shapeList.Trim({" "c, ";"c})

        union.in_features = shapeList           ' sets input features
        union.join_attributes = "ONLY_FID"      ' other options: "ALL", "NO_FID"
        union.out_feature_class = bufUnion      ' sets output feature class

        gp.Execute(union, Nothing)              ' executes the tool

        union = Nothing
        My.ArcMap.Application.StatusBar.Message(0) = "Buffers united."
    End Sub
```

Step 7, create a sub procedure named ClipOutput() to clip the merged shapefile by a user specified shape layer. The following code is straightforward.

```
Private Sub ClipOutput()
    My.ArcMap.Application.StatusBar.Message(0) = "Clipping buffers..."
    gp.AddOutputsToMap = True

    Dim clip As New Clip
    clip.in_features = bufUnion
    clip.clip_features = cboClipLayer.Text
    clip.out_feature_class = outShapefile

    gp.Execute(clip, Nothing)

    clip = Nothing
    My.ArcMap.Application.StatusBar.Message(0) = "Buffers clipped."
End Sub
```

Step 8, create a sub procedure DeleteTempFiles() to delete temporary shapefiles. This sub procedure uses a delete tool in the data management toolbox to delete each temporary file. Since the Delete class is in ESRI.ArcGIS.DataManagementTools assembly, you must add this assembly into your procedure as a reference if it is not added.

```
Private Sub DeleteTempFiles()
    My.ArcMap.Application.StatusBar.Message(0) = "Deleting temporary files..."
    gp.SetEnvironmentValue("workspace", "D:\Data\gpwks")

    Dim del As New ESRI.ArcGIS.DataManagementTools.Delete()

    del.in_data = bufUnion      ' to delete the merged shapefile
    gp.Execute(del, Nothing)

    For i As Integer = 0 To tmpBufs.Length - 1  ' to delete individual buffer files
        del.in_data = tmpBufs(i)
        gp.Execute(del, Nothing)
    Next

    del = Nothing
    My.ArcMap.Application.StatusBar.Message(0) = "Temporary files deleted."
End Sub
```

The DataManagementTools assembly and the ESRI.ArcGIS.AnalysisTools assembly each provides a clip tool, but the two clip tools are different. The data management clip tool is for clipping raster datasets while the analysis clip tool is for clipping vector features. To avoid conflict, you should not import both namespaces simultaneously in the same code module. To use a tool in an assembly whose namespace is not imported into the module, you can refer to the full namespace. For example, the following code is used to instantiate a delete object from the Delete class in the DataManagementTools assembly.

```
Dim del As New ESRI.ArcGIS.DataManagementTools.Delete()
```

Finally, after you have created custom procedures for all tasks, you can call them one after another in the button click event procedure as the following:

```
Private Sub btnBuffer_Click(...) Handles btnBuffer.Click
    ' 1. validate user input
    If Not ValidateUserInput() Then Return

    ' 2. initialize geoprocessing environment
    InitializeGeoprocessing()

    ' 3. get a feature layer object based on user selected layer name
    Dim layer As IFeatureLayer = GetLayerByName(cboInLayer.Text)

    ' 4. select features based on user-specified criterion
    Dim n As Integer = SelectFeatures(layer, cboFields.Text, txtAttVal.Text)
    If n <= 0 Then MsgBox("No feature selected") : Return ' no feature selected

    ' 5. create multiple buffer shapefiles
    CreateBuffers(layer, CInt(txtBufNum.Text), CInt(txtInterval.Text),
                  chkDissolveBuffers.Checked)

    ' 6. union multiple buffer shapefiles into one
    UnionBuffers()

    ' 7. clip the united buffer shapefile
    ClipOutput()

    ' 8. delete temporary files depending on user selection
    If chkDelTempFiles.Checked Then
        DeleteTempFiles()
    End If

    My.ArcMap.Application.StatusBar.Message(0) =
        "Creating buffers completed successfully!"

    If chkAutoClose.Checked Then    ' if user checked this checkbox
        Me.Close()                  ' close this window
    End If
End Sub
```

As a tip, the FormClosing event procedure of a Windows form is where you can perform clean up tasks. For example, you can add a line in this event to recycle the gp object as the following:

```
Private Sub frmBuffer_FormClosing(...) Handles Me.FormClosing
    gp = Nothing
End Sub
```

If you see no syntax error in your code after implementing all the procedures, you can run the program and test the buffer tool.

5.2 Spatial Analyst Tools

This part will show you two raster-based analysis tools: the slope tool and contour tool. The ESRI.ArcGIS.SpatialAnalystTools .NET assembly provides tools for raster-based analysis. To

use a tool in this assembly (toolbox), you must add the assembly into your project as a reference. In ArcGIS 10.2, this assembly is organize under the "Engine Extension" category in the reference page. Please make sure "Extension" is checked to load the assembly (Figure 5).

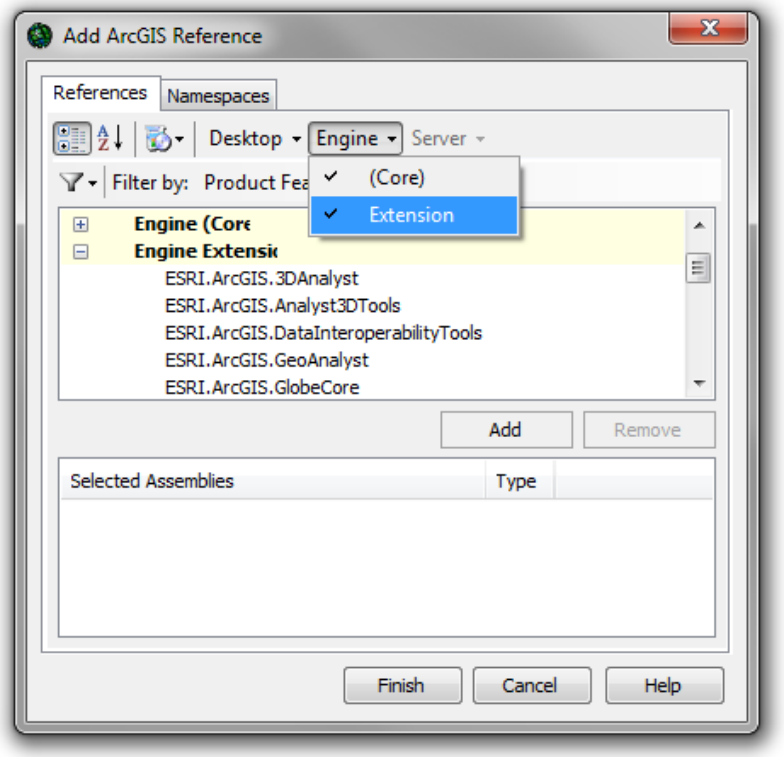


Figure 5

5.2.1 Slope tool

The SlopeButton add-in button in the project is intended for creating a slope raster from a DEM in the file geodatabase in the workspace. To implement the button, please add the .NET assembly ESRI.ArcGIS.SpatialAnalystTools into your project as reference if you have not done it. Then import the ESRI.ArcGIS.Geoprocessor and ESRI.ArcGIS.SpatialAnalystTools namespaces into the add-in button's class module. Finally use the following code to implement the OnClick event procedure.

```
Protected Overrides Sub OnClick()
    Dim gp As New Geoprocessor()     ' instantiates a geoprocessor object
    gp.SetEnvironmentValue("workspace", "D:\Data\gpwks\Meniscus.gdb")
    gp.OverwriteOutput = True

    Dim slope As New Slope()         ' instantiates a slope tool object
    slope.in_raster = "dem"          ' input. "dem" is the name of a DEM in the gdb
    slope.out_raster = "slope"       ' output. "slope" is the output raster name

    gp.Execute(slope, Nothing)       ' executes tool

    MsgBox("Slope raster created!")
End Sub
```

5.2.2 Contour tool

The ContourButton add-in button in the project is intended for creating a contour feature class from a DEM in the file geodatabase in the workspace. To implement this button, please first import namespaces ESRI.ArcGIS.Geoprocessor and ESRI.ArcGIS.SpatialAnalystTools into the contour add-in button's class module, and then implement the OnClick event procedure as the following.

```
Protected Overrides Sub OnClick()
    Dim gp As New Geoprocessor()        ' instantiates a Geoprocessor object
    gp.SetEnvironmentValue("workspace", "D:\Data\gpwks\Meniscus.gdb")  ' workspace
    ' set an output coordinate system
    gp.SetEnvironmentValue("outputCoordinateSystem", "Output")
    gp.OverwriteOutput = True

    Dim cont As New Contour()                       ' instantiates a Contour object
    cont.in_raster = "dem"                          ' sets input
    cont.out_polyline_features = "Output/contour"   ' sets output
    cont.base_contour = 2075                        ' sets base contour
    cont.contour_interval = 5                       ' sets contour interval

    Try
        gp.Execute(cont, Nothing)
        MsgBox("Contour done!")
    Catch ex As Exception
        MsgBox(ex.Message)
    End Try
End Sub
```

In the file geodatabase, the input DEM is projected to NAD_1983_UTM_Zone_12N coordinate system, but the Output feature dataset has a spatial reference of NAD_1927_UTM_Zone_12N. By default, a contour feature class produced by the tool will have the same coordinate system as the input DEM. In order to correctly write the resulting contour feature class in the Output feature dataset, a coordinate transformation must be performed. This is done by setting the "Output" feature dataset name to the outputCoordinateSystem" environment parameter.

After practicing with several geoprocessing tools, you can virtually use any tool available in the ArcToolbox by VB programming. By combining existing geoprocessing tools with the powerful user interface design capability of VB, you can now create tools to implement complex spatial analysis workflows and automate complex geoprocessing tasks using VB programming.

5.3 Data Management Tools

The Data Management Toolset in the ArcToolbox contains numerous tools organized in two dozens of categories. For add-in programming, tools in this toolbox are provided in the ESRI.ArcGIS.DataManagementTools assembly. Please add the assembly into your project as a reference if the assembly is not yet loaded. While implementing the DataButton add-in button in this example project, you will learn a few new ArcObjects components including the CreateFileGDB class, the CreateFeatureDataset class, and the CopyFeatures class. You will also learn the ListFeatureClasses method of the Geoprocessor class.

The gpwks folder on the CD-ROM contains several shapefiles. You will make the DataButton add-in button to perform three data management tasks at one click: 1) create a new file geodatabase; 2) create a feature dataset in the geodatabase; and 3) import all shapefiles into the geodatabase. To get started, please import the following namespaces into the DataButton class module and perform the steps below:

```
Imports ESRI.ArcGIS.Geoprocessor
Imports ESRI.ArcGIS.DataManagementTools
```

Step 1: Create a new file geodatabase. Use the following code to accomplish the task in the OnClick() event procedure.

```
    Dim gp As New Geoprocessor              ' instantiates a Geoprocessor objct
    Dim gdbCreator As New CreateFileGDB ()  ' instantiates a CreateFileGDB object

    gdbCreator.out_folder_path = "D:\Temp"  ' output folder, make sure this folder exists
    gdbCreator.out_name = "test"            ' sets new geodatabase name, must not exist
    gdbCreator.out_version = "CURRENT"      '
    Try        ' to avoid crashing caused by unexpected conditions
        gp.Execute(gdbCreator, Nothing)         ' execute the tool
        MsgBox("File geodatabase has been created.")
    Catch ex As Exception
        MsgBox("Creating file geodatabase failed.")
        Return
    End Try
```

Step 2: Create a new feature dataset inside the geodatabase with the following code below the code created by Step 1.

```
    Dim fdsCreator As New CreateFeatureDataset()            ' creates a tool object
    fdsCreator.out_dataset_path = "D:\Temp\test.gdb"        ' sets target geodatabase
    fdsCreator.out_name = "UTM12N"                          ' new feature dataset name
    fdsCreator.spatial_reference = "D:\Data\gpwks\Canopy.shp"  ' a spatial reference

    Try
        gp.Execute(fdsCreator, Nothing)         ' execute the tool
        MsgBox("Feature dataset created.")
    Catch ex As Exception
        MsgBox("Creating feature dataset failed.")
        Return
    End Try
```

The spatial reference property of a geoprocessing tool is flexible. This is demonstrated again in the fourth line that sets a spatial reference to the new feature dataset. In this example, a shapefile name is set to the property so that the spatial reference of the shapefile is be used. Make sure specified shapefile exists. Other acceptable values to set to the spatial reference property include a raster dataset name, a map projection file name, a block of text representing the content of a projection file, etc. For example,

```
    fdsCreator.spatial_reference = "D:\Data\gpwks\Slope"    ' suppose slope is a raster
    fdsCreator.spatial_reference = "D:\Data\gpwks\Canopy.prj"  ' a projection file
```

When you set a spatial reference to a new feature dataset, make sure you choose a correct one that matches the spatial reference of the data you will import.

Now, you can test the add-in button to see if it can create a file geodatabase in the specified location and create a feature dataset in it. After a successful test, please delete the database using ArcCatalog to get ready for future tests. If the geodatabase does not show in ArcCatalog, you may need to refresh the folder that contains the database in ArcCatalog.

Step 3: Import data into the feature dataset in the geodatabase. Suppose you receive a String array containing shapefile names derived from a custom function, you can write a loop to iterate through each shapefile and copy it into the geodatabase. At this point, you do not have to be concerned about the custom function that produce the file name array. You may simply create a blank function as the following and assume it to work perfect so that you can focus on file importing task.

```
Private Function GetShapefiles(folder As String) As String()

End Function
```

The function takes a folder argument and returns an array of shapefile names. Assuming this function works, you can call the function using the following two code lines at the bottom of the Click event procedure of the add-in button (DataButton). Make sure the folder containing the shapefiles exists on your computer.

```
' get all shapefiles in the folder and store them in a string array
Dim files() As String = GetShapefiles("D:\Data\gpwks")
If files Is Nothing Then Return    ' if the folder contains no feature classes
```

Then you can set the new geodatabase as the current workspace and instantiate a copy tool from the CopyFeatures class.

```
gp.SetEnvironmentValue("workspace", "D:\Temp\test.gdb")  ' sets current workspace
gp.AddOutputsToMap = False          ' suppress display of layers into ArcMap
Dim cp As New CopyFeatures()        ' instantiates a CopyFeatures object
```

Now, use the following repetition block to copy each shapefile into the geodatabase:

```
Dim success As Boolean = True               ' uses a variable to monitor copy status
For i As Integer = 0 To files.Length - 1    ' iterates through all files in the array
    cp.in_features = "D:\Data\gpwks\" & files(i)  ' sets an input shapefile
    cp.out_feature_class = "UTM12N\" & TakeOffExtension(files(i))  ' sets destination
    Try
        gp.Execute(cp, Nothing)     ' executes the copy tool
    Catch ex As Exception
        success = False             ' copying a shapefile failed
    End Try
Next
If success Then     ' copying process never fails
    MsgBox("Shapefiles successfully copied into geodatabase.")
End If
```

The code above uses a Boolean variable (*success*) to monitor copying status. The variable is initially assigned value True. If in any iteration the file copying code (gp.Execute()) fails and an error is caught by the Try block, variable *success* is turned False.

Since each shapefile name contains a file extension ".shp", while setting the out_feature_class parameter, i.e. to specify a destination feature class name in the geodatabase, the extension needs to be taken off. This is done by a custom function named TakeOffExtension() below. Please enter the following function into the DataButton class.

```
Private Function TakeOffExtension(filename As String) As String
    Return filename.Substring(0, filename.Length - 4)   ' removes the extension
End Function
```

If you test the program now, it only creates the file geodatabase because the GetShapefiles() function is not implemented so it does not really give you the shapefiles in a folder you specify. If the test produces a geodatabase, delete it using ArcCatalog.

Finally, let's implement the GetShapefiles() function. The key to this function is the a method of the Geoprocessor object named ListFeatureClass(wildCard, featureType, dataset). This method has three string type parameters. The first parameter wildcard can be used to set a filter to list feature class names containing certain characters. The wildcard is an asterisk symbol (*). For example, if you want to list all shapefiles whose names start with "Soil", you can pass a filter string "Soil*" to the wildcard parameter so that shapefiles such as "Soil01.shp", "Soil02.shp", and "Soil_clip.shp" will be listed. If you pass an empty pair of double quotes "" (no space in between) to the wildcard parameter, all feature classes will be listed.

The second parameter requires a string value to indicate the feature types to list. Table 4 lists all valid feature type values for this parameter. A shapefile is a simple feature so you can set "esriFTSimple" to the feature type parameter. If you set a pair of empty double quotes "" (no space) to this parameter, all feature classes will be listed. Generally, you can use this option.

Table 4 Feature Types[7]

Constant	Value	Description
esriFTSimple	1	Simple Feature.
esriFTSimpleJunction	7	Simple Junction Feature.
esriFTSimpleEdge	8	Simple Edge Feature.
esriFTComplexJunction	9	Complex Junction Feature.
esriFTComplexEdge	10	Complex Edge Feature.
esriFTAnnotation	11	Annotation Feature.
esriFTCoverageAnnotation	12	Coverage Annotation Feature.
esriFTDimension	13	Dimension Feature.
esriFTRasterCatalogItem	14	Raster Catalog Item.

The third parameter allows you to specify a dataset in a workspace. If the workspace is a geodatabase, you may specify a feature dataset so that feature classes in the specific dataset

[7] http://help.arcgis.com/en/sdk/10.0/arcobjects_net/componenthelp/index.html#/esriFeatureType_Constants/00250000004n000000/

will be listed. If the workspace is a folder, you may specify a subfolder as a dataset. Otherwise, you can set a pair of empty quote signs to this argument.

The ListFeatureClass() method returns a GpEnumList object with IGpEnumList interface. This interface is declared in namespace ESRI.ArcGIS.Geoprocessing which is the traditional COM based library. The GpEnumList object is simply a container that holds string values. This object provides a method named Next() to retrieve one value from it at a time. To retrieve all values from this object, you can use a repetition block to repeatedly call the Next() method. When no value is available after some iterations, the Next() method returns an empty string "" so that you can stop the repetition. If you want to re-read values from the GpEnumList object from beginning, you can use the Reset() method to restore the pointer to the first value. The GpEnumList object does not have a property to indicate the number of stored in it. Once the end of the list is reached, the Next() method returns an empty string. To get all shapefile names in a folder, you can use the following two lines to accomplish the task.

```
gp.SetEnvironmentValue("workspace", <a folder>)
Dim fileList As ESRI.ArcGIS.Geoprocessing.IGpEnumList =
                gp.ListFeatureClasses("", "", "")
```

Finally, load ESRI.GIS.Geoprocessing package into the project as a reference and then implement the GetShapefiles() function as the following:

```
Private Function GetShapefiles(folder As String) As String()
    Dim gp As New Geoprocessor()
    gp.SetEnvironmentValue("workspace", folder)

    Dim filenames() As String = Nothing  ' declares a string array to hold results
    Dim counter As Integer = 0           ' declares an integer counter

    ' call the ListFeatureClass method of Geoprocessor
    ' to get shapefile names in the workspace
    Dim fileList As ESRI.ArcGIS.Geoprocessing.IGpEnumList =
        gp.ListFeatureClasses("", "", "")

    ' the following populates the filenames() array with contents in fileList
    Dim Str As String = fileList.Next()     ' gets the first value from the list
    Do While Str <> ""                      ' starts a repetition
        ReDim Preserve filenames(counter)   ' expands the array
        filenames(counter) = Str            ' stores a value in the array
        counter += 1                        ' increments the counter
        Str = fileList.Next()               ' reads the next value from the list
    Loop

    Return filenames    ' returns a string array to the client
End Function
```

Start the program and test the DataButton add-in button. If you did every step correctly, you should find a file geodatabase in the workspace and all shapefiles imported into the specified feature dataset.

Summary and Highlights

In ArcGIS, geoprocessing tools are organized in the ArcToolbox. To use a tool, you can find its name in the ArcToolbox and then double click to open the tool's dialog box to find more information about the tool, including its usage and parameters. For detailed information of some geoprocessing tools, you may need to refer to the ArcGIS Resource Center.

In ArcObjects programming, geoprocessing tools are managed by toolboxes. Each geoprocessing system toolbox is represented by a managed .NET assembly. In order to use a geoprocessing tool, you must add the assembly that contains the tool into your project as a reference. For example, ordinary vector-based spatial analysis tools such as overlay analysis, buffering, and clipping are managed by the ESRI.ArcGIS.AnalysisTools assembly.

A geoprocessing tool can be instantiated from a tool class. Parameters of a tool are treated as properties of the tool object. To execute a tool, you must instantiate a geoprocessor object from the Geoprocessor class and call its Execute() method. Once you instantiate a geoprocessor object, you can use it to set up a geoprocessing environment such as to specify a current workspace, a geoprocessing extent, and an output coordinate system. You can also use the ListFeatureClass() method of a geoprocessor object to list feature classes in specified workspaces.

You can use tools in the data management toolset (in ESRI.ArcGIS.DataManagementTools assembly) to manage spatial data sets. For example, you can instantiate an object from the CreateFileGDB class to create file geodatabases, you can instantiate an object from the CreateFeatureDataset class to create feature datasets in a geodatabase, you can instantiate a file copying object from the CopyFeatures class to copy spatial data sets, and you can create a deletion object from the Delete class to delete spatial data sets.

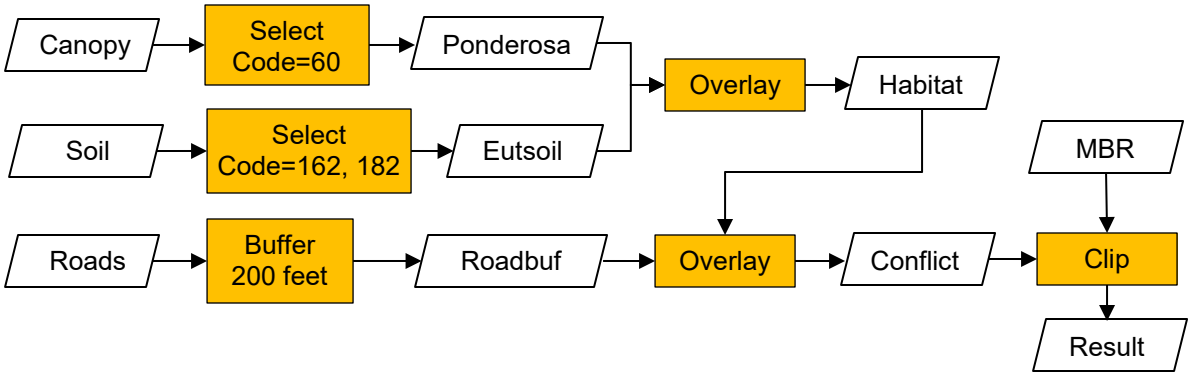
Exercises

1. Answer the following questions.
   1) How do you find the name of a geoprocessing tool and its parameter information?
   2) Do you get to the website of the online geoprocessing tool reference?
   3) How to you decide which .NET assembly to add into your project as a reference in order to use a specific geoprocessing tool?
   4) How to you find which parameters are required and which are optional for a tool?
   5) Values set to parameters of a geoprocessing tool are always strings. True or false?
   6) What is the main difference between ESRI.ArcGIS.Geoprocessing and ESRI.ArcGIS.Geoprocessor?
   7) What is the difference between GeoProcessor class and Geoprocessor class?
   8) What does a Geoprocessor object do?
   9) What method of the Geoprocessor object do you call to set a geoprocessing environment?
   10) While setting a geoprocessing environment, how to you find a valid environment name?
   11) In how many different ways can you set a geoprocessing extent environment?
   12) In how many different ways can you set a geoprocessing output coordinate system environment?
   13) What method of the Geoprocessor object do you call to execute a tool?
   14) That is a CancelTracker object?
   15) What property of the Geoprocessor object do you set to allow/disallow geoprocessing tools to over write output files?
   16) What property of the Geoprocessor object do you set to allow/disallow geoprocessing tools to display any resulting data in ArcMap?
   17) How do you use geoprocessing tools in VB to create a new file geodatabase?
   18) How do you use geoprocessing tools in VB to get a list of feature classes in a workspace?
   19) How do you use geoprocessing tools to import shapefiles into a geodatabase?
   20) How do you use geoprocessing tools to delete data such as shapefiles?

2. A hypothetical endangered specious *meniscus* is found associated with ponderosa pine forest and shallow eutroboralf soil in a project area. A major threat to this plant is human activities, especially recreational activities along roads. To protect this plant, we need to identify possible habitats of this plant as well as potential conflict locations with human activities.

   A file geodatabase (Data\gpwks\Meniscus.gdb on CD-ROM) has been created for a meniscus protection project. It contains necessary feature classes for this project including Canopy, Soil, Roads, and MBR (minimum bounding rectangle). Ponderosa pine forest is represented by code 60 in the Canopy feature class, and eutroboralf soil is represented by codes 162 and 182 in the Soil feature class. Conflict can be defined as a zone of 200 feet on each side of all roads. The following flowchart shows a procedures of spatial analyses to perform to identify potential habitats and conflict locations.

   Create an ArcMap add-in project named “Meniscus” and one add-in button in the project that calls existing geoprocessing tools to accomplish all the analysis tasks.


Canopy
Select Code=60
Ponderosa
Soil
Select Code=162, 182
Eutsoil
Overlay
Habitat
MBR
Roads
Buffer 200 feet
Roadbuf
Overlay
Conflict
Clip
Result